LogCDM: Interpretable Log Anomaly Detection via Behavioral Graph Drift and Adaptive Signal Fusion

Suryansh Talukdar *talusb01@pfw.edu*
Purdue University Fort Wayne

Taha Sarwat *sarwt01@pfw.edu*
Purdue University Fort Wayne

Jin Soung Yoo *jyoo@pfw.edu*
Purdue University Fort Wayne

arXiv:2611.12775v1 [cs.AI] 9 Nov 2026

Abstract

Log-based anomaly detection is increasingly dominated by large pretrained models, from BERT-scale encoders to 8B-parameter language models, which need GPUs at inference time and are hard to deploy on the monitoring nodes where logs are produced. We present LogCDM, a compact detector that scores each log session with three complementary signals: the next-event surprise of a small causal Transformer, the distance between a graph-attention embedding of the session's own transition graph and that of a normal reference graph, and a transition-drift statistic that we show is exactly a frequency-weighted χ^2 divergence between transition kernels. The divergence view yields an asymptotic null distribution, a length-standardized score, and a per-edge decomposition that explains each alarm. A sigmoid gate fuses the three signals; under a shared-covariance Gaussian model this gate is the Bayes posterior, and its learned weights indicate which signal each dataset relies on. With 8.1–8.8M parameters and no GPU at inference, LogCDM reaches F1 of 0.984 on HDFS, 0.870 on chronologically split BGL and 0.850 on Thunderbird, within 0.013–0.12 of an 8.1B-parameter LLM detector evaluated under the same protocol and above all compact baselines on BGL. It scores a session in 5.5–10.5 ms on a CPU. Ablations show that the structural signals add 0.025–0.030 F1 on BGL and Thunderbird but almost nothing on HDFS, consistent with the learned fusion weights.

1 Introduction

Modern distributed systems, from Hadoop pipelines to supercomputers and cloud microservices, record their runtime behavior in system logs. Hardware faults, stalled jobs and cascading service failures all leave traces in the order and structure of logged events, and the volume of these logs rules out manual inspection (He et al., 2021; Landauer et al., 2023). Log-based anomaly detection (LAD) asks whether a session or window of log events deviates from normal behavior, given historical logs and, in the semi-supervised setting, a small set of labeled anomalies.

Three properties make LAD hard. Raw log lines are free text whose surface form changes across software versions, so detectors first parse them into templates (He et al., 2017), and parsing errors bound what any downstream model can achieve (Shin et al., 2021). Anomalies are rare, from about 3% of HDFS sessions to about 1% of Thunderbird windows, which makes threshold-based decisions fragile (Le & Zhang, 2022). Finally, anomalies are heterogeneous: some insert a forbidden event, some change event counts, and some change how events follow one another across a whole session without introducing any unusual event locally (Meng et al., 2019; Li et al., 2024).

Limitations of prior work. Most deep LAD methods score a session by how surprising its event sequence is under a model of normal sequences. DeepLog uses an LSTM next-event predictor (Du et al., 2017);

LogRobust and NeuralLog use semantic template embeddings with recurrent or Transformer encoders (Zhang et al., 2019; Le & Zhang, 2021); LogBERT and LogFormer adapt masked-language-model pretraining (Guo et al., 2021; 2024). Recent detectors push accuracy further with large language models: LogLLM combines BERT with Llama-3-8B (Guan et al., 2024), CoLA routes hard cases from a small model to a fine-tuned 8B LLM (Zhu et al., 2025), and LogLLaMA fine-tunes a LLaMA-2 architecture with reinforcement learning (Yang & Harris, 2025). This trend leaves two gaps.

First, *cost*. LLM-based detectors need a GPU to score each session and gigabytes of memory to hold the model, while the nodes that most need monitoring (edge devices, cluster managers, on-premise servers) often have neither. Evaluation studies have long argued that time and resource cost belong next to accuracy when judging LAD systems (Le & Zhang, 2022; Xiao et al., 2026), yet model size has grown by three orders of magnitude.

Second, *structure and explanation*. Sequence surprise is computed locally, one next event at a time. It can miss sessions whose events are individually plausible but whose overall pattern of transitions has shifted, and it gives the operator a single number with no account of which transitions caused the alarm. Graph-based detectors model transition structure (Li et al., 2024), but they usually replace sequence models instead of combining with them, and their scores lack a statistical interpretation.

Our approach. LogCDM (Log Contextual Dual-pathway Model) combines three signals in one small model (Figure 1). A 6-layer causal Transformer gives a sequence score. A graph attention network (Veličković et al., 2018) embeds the session's own transition graph and compares it with an embedding of a normal reference graph. A closed-form Structural Transition Drift Score (STDS) compares the session's transition frequencies with the reference. A sigmoid gate fuses the three scores. Template embeddings are computed once offline, so the deployed model never calls a language model.

Contributions.

1. **A compact three-signal detector.** LogCDM has 8.1–8.8M parameters, roughly 1,000× fewer than LLM-based detectors, and runs on a CPU (Section 3).
2. **A statistical account of structural drift.** We prove that STDS equals a frequency-weighted χ^2 divergence between transition kernels, and equivalently a weighted norm of the difference of random-walk Laplacians. This yields a total-variation lower bound, a bounded score under smoothing, an asymptotic χ^2 null distribution, a length-standardized score, and a per-edge decomposition for explaining alarms (Section 4).
3. **An interpretable fusion gate.** Under a shared-covariance Gaussian model the gate is the Bayes posterior, so its standardized weights reflect each signal's class separation. We also show that a common way of feeding an adapted adjacency matrix to a GAT passes no gradient to the adapter, and give a support-preserving adapter that does (Section 4).
4. **A protocol-aware evaluation.** We evaluate on HDFS, BGL and Thunderbird with random and chronological splits, five seeds, AUC-PR, bootstrap intervals, CPU latency, nine ablations, label-noise tests and fusion-weight analysis (Section 5).

2 Related Work

Sequence models for log anomaly detection. DeepLog frames LAD as next-event prediction with an LSTM trained on normal sessions (Du et al., 2017). LogAnomaly adds template semantics and count features (Meng et al., 2019), LogRobust uses FastText embeddings with an attention Bi-LSTM to tolerate evolving log statements (Zhang et al., 2019), and Lu et al. (2018) use a CNN over embedded event sequences. NeuralLog skips parsing and encodes raw messages with BERT (Le & Zhang, 2021). LogBERT (Guo et al., 2021) trains a small Transformer with masked log-key prediction and a hypersphere objective, and LogFormer (Guo et al., 2024) pretrains on one system and adapts to another with adapters. LightLog targets edge

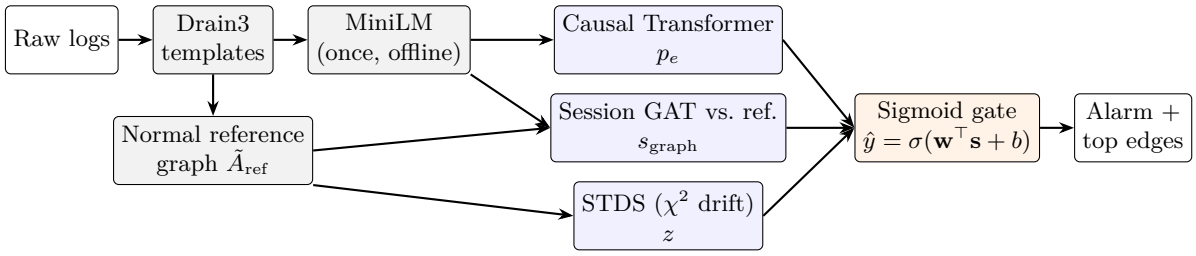


Figure 1: LogCDM overview. Grey boxes run once offline; blue boxes run per session on a CPU. The three scores are fused by a sigmoid gate, and the STDS term decomposes over transitions to explain each alarm.

devices with a temporal convolutional network (Wang et al., 2022), and LogMamba uses state-space models with a frequency branch (Hu et al., 2026). LogDA combines SBERT template vectors with dual attention for imbalanced data (Zhang & Fu, 2025). These models score order and content but not session-level transition structure, and none gives a statistical meaning to its score.

Large language models for logs. LogLLM feeds BERT message embeddings into Llama-3-8B and reports the strongest results under a common protocol (Guan et al., 2024). CoLA lets a mixture-of-experts Bi-LSTM defer uncertain sessions to a fine-tuned Llama3-8B (Zhu et al., 2025), and LogLLaMA fine-tunes a LLaMA-2 architecture with reinforcement learning (Yang & Harris, 2025). These methods set the accuracy frontier but require GPUs and billions of parameters at inference. LogCDM targets the opposite end of this trade-off.

Graph-based detection. Logs2Graphs converts sessions to attributed directed graphs and detects anomalies with a one-class digraph convolutional network, showing that transition topology carries signal beyond event order (Li et al., 2024). We keep this insight but embed each session's graph next to a sequence model, compare it with a reference graph, and add a closed-form divergence with known statistical properties. Pearson-type tests on Markov transition counts go back to Anderson & Goodman (1957); our contribution is to connect that classical test to a learned detector, use it for length standardization and explanation, and fuse it with neural scores.

Parameter-efficient adaptation and score fusion. LoRA adapts frozen weights with low-rank updates (Hu et al., 2022), and GraphLoRA brings low-rank adaptation to graph neural networks (Yang et al., 2024). We use a low-rank, support-preserving correction of the reference adjacency matrix instead of the network weights. Our two-phase training, normal-only pretraining followed by a small labeled set, follows semi-supervised anomaly detection (Ruff et al., 2020; 2021), and our fusion gate is the discriminative counterpart of linear discriminant analysis (Hastie et al., 2009; Ng & Jordan, 2002).

Lightweight and explainable detection. Compact detectors for constrained hardware include LightLog's temporal convolutions (Wang et al., 2022) and LogFormer's adapter tuning, which trains about 0.4M–1M parameters (Guo et al., 2024). Neither reports CPU latency together with accuracy under a chronological protocol, which is the comparison Xiao et al. (2026) argue for. Explanations for log anomalies are usually post hoc, for example subgraph explanations of a graph detector's output (Li et al., 2024). In LogCDM the explanation is part of the score: the STDS term decomposes exactly over transitions, so the evidence behind an alarm is available without an extra explainer.

Evaluation practice. Random splits leak future information and inflate scores, while chronological splits lower them and can reorder methods (Le & Zhang, 2022; Xiao et al., 2026). Common benchmarks also contain sessions that are trivially separable (Landauer et al., 2024). We therefore report chronological splits, AUC-PR (Saito & Rehmsmeier, 2015), seed variance and bootstrap intervals, and separate same-protocol from different-protocol comparisons.

3 Methodology

3.1 Problem setup

Definition 1 (Sessions and vocabulary)**.** A parser π maps each log line to a template in a finite vocabulary $\mathcal{V} = \{1, \dots, V\}$, with 0 reserved for padding. A session is a sequence $x^{(i)} = (e_1^{(i)}, \dots, e_T^{(i)}) \in (\mathcal{V} \cup \{0\})^T$ grouped by block ID (HDFS) or by a fixed window of log lines (BGL, Thunderbird), with label $y^{(i)} \in \{0, 1\}$.

Definition 2 (Leakage-free partition)**.** Sessions are split into disjoint sets $\mathcal{I}_{\text{tr}}, \mathcal{I}_{\text{val}}, \mathcal{I}_{\text{te}}$, either chronologically or by stratified random sampling. The parser, vocabulary, embedding projection, reference graph, score normalization constants and decision threshold are all estimated from $\mathcal{I}_{\text{tr}}$ and $\mathcal{I}_{\text{val}}$ only. Templates first seen in $\mathcal{I}_{\text{te}}$ map to an unknown token.

The task is to learn a score $f : x \mapsto [0, 1]$ and threshold $\tau^\star$ such that $\hat{y} = \mathbb{1}[f(x) > \tau^\star]$ maximizes F1 on $\mathcal{I}_{\text{val}}$; we report performance on $\mathcal{I}_{\text{te}}$. Training uses normal sessions $\mathcal{S}_N \subset \mathcal{I}_{\text{tr}}$ and a labeled fusion set $\mathcal{S}_F \subset \mathcal{I}_{\text{tr}}$ containing a fraction of the training sessions with their labels.

3.2 Preprocessing

Parsing and embeddings. Drain3 (He et al., 2017) is fitted on training logs. Each template's text is encoded once with all-MiniLM-L6-v2 (Reimers & Gurevych, 2019; Wang et al., 2020) into $\mathbb{R}^{384}$ and mapped to $d = 256$ dimensions, by PCA when the vocabulary is large enough (BGL, Thunderbird) and by a learned linear projection otherwise (HDFS has only 39 templates). The result $E \in \mathbb{R}^{V \times d}$ initializes the Transformer's embedding table and the GAT node features. The sentence encoder is never called during training or inference.

Transition counts. For a session i and maximum lag L, let

$$N_i(u, v) = \sum_{\ell=1}^{L} \sum_{t=1}^{T-\ell} \mathbb{1}[e_t^{(i)} = u]\, \mathbb{1}[e_{t+\ell}^{(i)} = v], \qquad n_i(u) = \sum_v N_i(u, v), \qquad n_i = \sum_u n_i(u). \tag{1}$$

Let $U_i = \{u : n_i(u) > 0\}$ be the source events the session visits, $\pi_i(u) = n_i(u)/n_i$ their empirical frequencies, and $P_i(v \mid u) = N_i(u, v)/n_i(u)$ the session's transition kernel. The reference counts A_{ref} sum Eq. equation 1 over $\mathcal{S}_N$, and P_{ref} is their row normalization.

Smoothed, coarsened reference. For each source u, let $\Omega_u = \operatorname{supp} P_{\text{ref}}(\cdot \mid u) \cup \{\varnothing\}$, where $\varnothing$ pools all transitions never seen from u in normal data. We define

$$\tilde{P}_{\text{ref}}(\omega \mid u) = (1 - \lambda)\, P_{\text{ref}}(\omega \mid u) + \lambda \mathbb{1}[\omega = \varnothing], \qquad \omega \in \Omega_u, \tag{2}$$

with $\lambda = 10^{-3}$, and map $P_i(\cdot \mid u)$ onto Ω_u in the same way. Every cell of $\tilde{P}_{\text{ref}}$ is then positive, and a novel transition counts as mass on $\varnothing$ instead of an infinite deviation.

3.3 Three anomaly signals

Sequence score. A causal Transformer encoder (Vaswani et al., 2017) with $N_{\text{layer}} = 6$ pre-norm layers, model width $d = 256$, 8 heads and feed-forward width $d_{\text{ffn}} = 2048$ maps the embedded, positionally encoded session to contexts $C^{(i)} \in \mathbb{R}^{T \times d}$. A linear head predicts the next template, and the sequence score is the mean next-event cross-entropy over non-padding targets:

$$p_e^{(i)} = \frac{\sum_{t=1}^{T-1} \mathbb{1}[e_{t+1} \neq 0]\; \ell_{\text{CE}}\big(W_{\text{head}} C_t^{(i)},\, e_{t+1}\big)}{\sum_{t=1}^{T-1} \mathbb{1}[e_{t+1} \neq 0]}. \tag{3}$$

Structural Transition Drift Score. STDS measures how far the session's transition kernel lies from the reference, weighted by how often each source event occurs:

$$s_i^2 = \sum_{u \in U_i} \pi_i(u) \sum_{\omega \in \Omega_u} \frac{\big(P_i(\omega \mid u) - \tilde{P}_{\text{ref}}(\omega \mid u)\big)^2}{\tilde{P}_{\text{ref}}(\omega \mid u)}. \tag{4}$$

Section 4 shows that $n_i s_i^2$ is Pearson's statistic for the hypothesis that the session follows the reference kernel. Because its null mean grows with the number of cells and shrinks with session length, raw s_i^2 is biased toward short sessions. We therefore feed the fusion gate the standardized score

$$z_i = \frac{n_i s_i^2 - d_i}{\sqrt{2d_i}}, \qquad d_i = \sum_{u \in U_i} \big(|\Omega_u| - 1\big), \tag{5}$$

which has approximately zero mean and unit variance on normal sessions of any length (Corollary 1).

Session-graph score. Let A_i be the session's transition-count matrix restricted to its visited events. A two-layer graph attention network with edge-weighted attention,

$$\alpha_{uv} = \frac{A[u,v] \exp\big(\mathrm{LeakyReLU}(\mathbf{a}^\top [W h_u \,\|\, W h_v])\big)}{\sum_k A[u,k] \exp\big(\mathrm{LeakyReLU}(\mathbf{a}^\top [W h_u \,\|\, W h_k])\big)}, \qquad h'_u = \mathrm{ELU}\Big(\sum_v \alpha_{uv} W h_v\Big), \tag{6}$$

encodes A_i into a mean-pooled, ℓ_2-normalized embedding g_i. The same network encodes the adapted reference graph $\tilde{A}_{\mathrm{ref}}$ into g_{ref}, and

$$s^{(i)}_{\mathrm{graph}} = \|g_i - g_{\mathrm{ref}}\|_2^2. \tag{7}$$

Weighting attention by $A[u,v]$ (rather than using A only to select neighbors) lets gradients reach the reference graph (Proposition 4).

Low-rank reference adapter. The reference graph is corrected multiplicatively on its existing edges:

$$\tilde{A}_{\mathrm{ref}} = A_{\mathrm{ref}} \odot \big(\mathbf{1}\mathbf{1}^\top + \beta \tanh(PQ^\top)\big), \qquad P, Q \in \mathbb{R}^{V \times r},\ r = 4,\ \beta = 0.1, \tag{8}$$

with Q initialized to zero. The adapter has $2Vr$ parameters (312 on HDFS, 10,400 on BGL), leaves the reference unchanged at initialization, keeps its support, and keeps every weight within $[1-\beta, 1+\beta]$ of its original value.

3.4 Fusion and training

Each score is min-max normalized with training-split constants, giving $\mathbf{s}^{(i)} = (\tilde{p}_e^{(i)}, \tilde{s}^{(i)}_{\mathrm{graph}}, \tilde{z}_i)^\top$, and

$$\hat{y}^{(i)} = \sigma\big(\mathbf{w}^\top \mathbf{s}^{(i)} + b\big), \qquad \mathbf{w} = (w_a, w_b, w_c)^\top. \tag{9}$$

Phase 1: normal-only pretraining. Only the Transformer is trained, on $\mathcal{S}_N \setminus \mathcal{S}_F$, to minimize the mean of p_e (Eq. 3). The checkpoint with the lowest cross-entropy on held-out normal validation sessions is kept.

Phase 2: labeled fusion. All parameters are trained on $\mathcal{S}_F$ with a class-balanced sampler and

$$\mathcal{L} = \mathcal{L}_{\mathrm{BCE}} + \lambda_1 \mathcal{L}_{\mathrm{rec}} + \lambda_2 \mathcal{L}_{\mathrm{cont}}, \qquad \lambda_1 = 0.15,\ \lambda_2 = 0.10, \tag{10}$$

where $\mathcal{L}_{\mathrm{BCE}}$ is binary cross-entropy on $\hat{y}$ with label smoothing 0.1, $\mathcal{L}_{\mathrm{rec}} = \mathbb{E}_{y=0}[p_e]$ keeps the Phase 1 sequence model close to its normal prior, and $\mathcal{L}_{\mathrm{cont}} = \max\big(0, \gamma - (\mathbb{E}_{y=1}[s_{\mathrm{graph}}] - \mathbb{E}_{y=0}[s_{\mathrm{graph}}])\big)$ with $\gamma = 0.5$ is a margin term in the spirit of Hadsell et al. (2006). After each epoch the threshold $\tau^\star$ maximizing validation F1 is computed over quantiles of validation scores, and the checkpoint with the best validation F1 is kept. Algorithm 1 summarizes training.

3.5 Inference

At deployment, g_{ref} and $\tilde{P}_{\mathrm{ref}}$ are cached. Each session needs one Transformer pass, a GAT pass over at most T nodes and the closed-form STDS. The cost is dominated by the Transformer, $O\big(N_{\mathrm{layer}}(Td\, d_{\mathrm{ffn}} + T^2 d)\big)$ per session, and the STDS costs $O(n_i)$ with sparse counts.

Algorithm 1 LogCDM training

Require: Training logs; labeled fusion set $\mathcal{S}_F$; validation set; lag L; smoothing λ

Fit Drain3 on training logs; build vocabulary; compute E once with MiniLM
Build A_{ref} from normal training sessions; form $\tilde{P}_{\text{ref}}$ by Eq. equation 2
Phase 1: train the Transformer on $\mathcal{S}_N \setminus \mathcal{S}_F$ to minimize $\mathbb{E}[p_e]$; keep best normal-validation checkpoint
Fit min-max constants for p_e, s_{graph}, z on training sessions
Phase 2: initialize $Q \leftarrow 0$; train all parameters on $\mathcal{S}_F$ with Eq. equation 10
  after each epoch: refit normalization constants; choose $\tau^\star$ on validation; keep best checkpoint
Cache g_{ref}; **return** model, $\tau^\star$, constants

4 Theoretical Analysis

This section states the properties of the structural score, the fusion gate and the adapter. Full proofs are in Appendix A.

4.1 STDS as a divergence between transition kernels

For distributions p, q on a finite set with $q > 0$, write $\chi^2(p\|q) = \sum_\omega (p(\omega) - q(\omega))^2/q(\omega)$ and $\text{TV}(p, q) = \frac{1}{2}\sum_\omega |p(\omega) - q(\omega)|$. The random-walk Laplacian of a row-stochastic matrix P is $\mathcal{L} = I - P$.

Theorem 1 (STDS is a weighted χ^2 divergence)**.** *Let $\tilde{p}_{\min} = \min_{u,\omega} \tilde{P}_{\text{ref}}(\omega \mid u) > 0$. For every session i:*

(a) Divergence form. $s_i^2 = \sum_{u \in U_i} \pi_i(u)\, \chi^2\big(P_i(\cdot \mid u) \,\|\, \tilde{P}_{\text{ref}}(\cdot \mid u)\big)$. *Hence $s_i \geq 0$, with equality if and only if $P_i(\cdot \mid u) = \tilde{P}_{\text{ref}}(\cdot \mid u)$ for every visited $u \in U_i$.*

(b) Laplacian form. *With $\mathcal{L}_i = I - P_i$ and $\tilde{\mathcal{L}}_{\text{ref}} = I - \tilde{P}_{\text{ref}}$ on the coarsened outcome spaces, $s_i^2 = \big\|\Pi_i^{1/2}(\mathcal{L}_i - \tilde{\mathcal{L}}_{\text{ref}}) \oslash \tilde{P}_{\text{ref}}^{1/2}\big\|_F^2$, where $\Pi_i = \text{diag}(\pi_i)$ and $\oslash$, $\cdot^{1/2}$ act entrywise.*

(c) Total-variation bound. $4 \sum_{u \in U_i} \pi_i(u)\, \text{TV}^2\big(P_i(\cdot \mid u), \tilde{P}_{\text{ref}}(\cdot \mid u)\big) \leq s_i^2$.

(d) Boundedness. $s_i^2 \leq 1/\tilde{p}_{\min} - 1$, *so no single novel transition can make the score unbounded.*

(e) Edge decomposition. $s_i^2 = \sum_{(u,\omega)} c_i(u, \omega)$ *with* $c_i(u, \omega) = \pi_i(u)\,(P_i(\omega \mid u) - \tilde{P}_{\text{ref}}(\omega \mid u))^2/\tilde{P}_{\text{ref}}(\omega \mid u) \geq 0$.

Proof sketch. Part (a) regroups Eq. equation 4 by source. Part (b) holds because $\mathcal{L}_i - \tilde{\mathcal{L}}_{\text{ref}} = \tilde{P}_{\text{ref}} - P_i$ entrywise, including the diagonal, so the Laplacian difference and the kernel difference have the same squared entries; this is why the earlier unnormalized-Laplacian formulation failed on self-loops and the random-walk form does not. Part (c) applies Cauchy–Schwarz, $\sum_\omega |p - q| \leq \sqrt{\chi^2(p\|q)}$, row by row. Part (d) uses $\chi^2(p\|q) = \sum_\omega p^2/q - 1 \leq 1/\min q - 1$ and $\sum_u \pi_i(u) = 1$. Part (e) is the definition. □

Part (e) is what makes LogCDM's alarms interpretable: the contributions $c_i(u, \omega)$ rank the transitions that drove a session's score (Section 5.7).

The next result formalizes the blind spot described in Section 1. Many sequence detectors, DeepLog with a top-g rule being the clearest case (Du et al., 2017), flag a session only if some observed transition is unlikely under the normal model. STDS also reacts when every transition is individually normal but their frequencies have shifted.

Proposition 1 (Frequency shifts invisible to support-based rules)**.** *Call a detector support-based if it flags a session only when the session contains a transition (u, v) with $P_{\text{ref}}(v \mid u) = 0$; a top-$g$ next-event rule with g at least the maximum out-degree of the reference graph is an example. Let u have at least two reference successors and let $v^\star$ be its least likely one, with $q^\star = \tilde{P}_{\text{ref}}(v^\star \mid u)$. Any session that visits u with frequency $\pi_i(u)$, always moves from u to $v^\star$, and otherwise follows the reference is never flagged by a support-based rule, yet*

$$s_i^2 \;\geq\; \pi_i(u)\Big(\frac{1}{q^\star} - 1\Big).$$

Proof sketch. Every transition lies in the reference support, so a support-based rule stays silent. Row u of the session is the point mass $\delta_{v^\star}$, and $\chi^2(\delta_{v^\star}\|q) = (1-q^\star)^2/q^\star + \sum_{\omega\neq v^\star} q(\omega) = 1/q^\star - 1$; the other rows contribute nonnegative terms. □

For a successor seen in 1% of normal transitions from u, the bound is $99\,\pi_i(u)$, close to the maximum allowed by Theorem 1(d). Repetition bursts of an otherwise normal event, as in the example of Section 5.7, have this form. A learned sequence model can still assign them some surprise, but nothing in its objective guarantees it; STDS reacts by construction.

4.2 Null distribution and length standardization

Proposition 2 (Asymptotic null distribution)**.** *Let $L = 1$. Suppose a session is generated by a Markov chain whose transition kernel, after coarsening to Ω_u, equals $\tilde{P}_{\mathrm{ref}}$, and let $\min_{u\in U_i} n_i(u) \to \infty$ with U_i fixed. Then*

$$n_i s_i^2 = \sum_{u\in U_i}\sum_{\omega\in\Omega_u} \frac{\big(N_i(u,\omega) - n_i(u)\tilde{P}_{\mathrm{ref}}(\omega \mid u)\big)^2}{n_i(u)\tilde{P}_{\mathrm{ref}}(\omega \mid u)} \xrightarrow{d} \chi^2_{d_i}, \qquad d_i = \sum_{u\in U_i} \big(|\Omega_u| - 1\big).$$

The identity follows from $\pi_i(u) = n_i(u)/n_i$; the limit is the Pearson goodness-of-fit theorem for Markov chains of Anderson & Goodman (1957).

Corollary 1 (Consistency and length standardization)**.** *Under the conditions of Proposition 2, $\mathbb{E}[s_i^2] \approx d_i/n_i$, so raw STDS is larger for short sessions even when they are normal, whereas z_i in Eq. equation 5 has asymptotic mean 0 and variance 1 for any length. If instead the session follows an ergodic chain with kernel $P \neq \tilde{P}_{\mathrm{ref}}$ on some visited row, so that $\delta = \sum_u \bar{\pi}(u)\chi^2(P(\cdot \mid u)\|\tilde{P}_{\mathrm{ref}}(\cdot \mid u)) > 0$ for the stationary frequencies $\bar{\pi}$, then $s_i^2 \to \delta$ almost surely, $z_i \to \infty$, and the test $\mathbb{1}[n_i s_i^2 > \chi^2_{d_i,1-\alpha}]$ has power tending to one.*

Remark 1. For lags $L > 1$ the counts overlap and are no longer conditionally multinomial, and for short sessions with large vocabularies the χ^2 approximation is loose. In these regimes we use z_i only as a length-normalized score and let the fusion gate calibrate it; we do not report p-values.

4.3 Fusion as a Bayes classifier

Proposition 3 (Fusion gate under a Gaussian score model)**.** *Suppose $\mathbf{s} \mid y \sim \mathcal{N}(\boldsymbol{\mu}_y, \Sigma)$ for $y \in \{0,1\}$ with shared $\Sigma \succ 0$ and prior $\rho = P(y=1)$. Then $P(y=1 \mid \mathbf{s}) = \sigma(\mathbf{w}^{\star\top}\mathbf{s} + b^\star)$ with*

$$\mathbf{w}^\star = \Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0), \qquad b^\star = -\tfrac{1}{2}\big(\boldsymbol{\mu}_1^\top\Sigma^{-1}\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0^\top\Sigma^{-1}\boldsymbol{\mu}_0\big) + \log\tfrac{\rho}{1-\rho}.$$

If $\Sigma = \mathrm{diag}(\sigma_a^2, \sigma_b^2, \sigma_c^2)$, the standardized weights satisfy $w_k^\star\sigma_k = \Delta\mu_k/\sigma_k$ for each signal k, the class separation of that signal.

This is the linear discriminant analysis posterior (Hastie et al., 2009). Logistic regression estimates $(\mathbf{w}^\star, b^\star)$ consistently whenever the log-odds are linear in $\mathbf{s}$, even without Gaussianity (Ng & Jordan, 2002). Min-max-normalized scores are bounded and the gate is trained jointly with the other losses, so we treat Proposition 3 as an interpretation of the learned weights and check it empirically (Section 5.5), not as a guarantee.

4.4 The reference adapter

Proposition 4 (Adapter identity, support and gradient)**.** *For the adapter in Eq. equation 8:*

- *(a) at initialization ($Q = 0$), $\tilde{A}_{\mathrm{ref}} = A_{\mathrm{ref}}$ exactly;*
- *(b) for all P, Q, $\operatorname{supp}\tilde{A}_{\mathrm{ref}} = \operatorname{supp} A_{\mathrm{ref}}$ and $(1-\beta)A_{\mathrm{ref}} \le \tilde{A}_{\mathrm{ref}} \le (1+\beta)A_{\mathrm{ref}}$ entrywise;*
- *(c) if a graph network depends on $\tilde{A}_{\mathrm{ref}}$ only through the neighborhood indicator $\mathbb{1}[\tilde{A}_{\mathrm{ref}} > 0]$, then $\partial\mathcal{L}/\partial P = \partial\mathcal{L}/\partial Q = 0$ for every loss $\mathcal{L}$;*
- *(d) with edge-weighted attention (Eq. 6), $\partial\alpha_{uv}/\partial\tilde{A}[u,w] = \alpha_{uv}(\mathbb{1}[v=w] - \alpha_{uw})/\tilde{A}[u,w]$, which is nonzero whenever node u has at least two neighbors.*

Table 1: Dataset statistics after grouping and splitting.

	HDFS	BGL	Thunderbird
Sessions / windows	575,061	47,100	100,000
Anomaly rate (%)	2.93	9.5	1.0
Train / val / test	402.5k / 57.5k / 115.0k	33.0k / 4.7k / 9.4k	70.0k / 10.0k / 20.0k
Test anomalies	3,367	800	–
Templates (fit on train)	39	1,300	1,076
Test OOV template rate (%)	0	3	4
Max length T / % truncated	64 / <5	100 / 0	100 / 0
Lag L	2	3	2
Fusion set / labeled anomalies	60,381 / 1,768	5,000 / 525	10,500 / 100

Part (c) explains a failure mode of thresholded adjacency inputs: by (b) the support never changes, so the indicator is constant and the adapter never learns. Part (d) shows that edge weighting removes this failure. At $Q = 0$ the gradient with respect to Q is $\beta\,(\partial\mathcal{L}/\partial\tilde{A} \odot A_{\text{ref}})^{\top} P$, which is nonzero for generic P, so training starts from the exact reference and moves away from it only as the loss requires.

4.5 Complexity

Proposition 5 (Inference cost)**.** *With g_{ref} cached, scoring one session costs $O\big(N_{\text{layer}}(Td\,d_{\text{ffn}} + T^2 d) + TdV\big)$ for the Transformer and prediction head, $O(|U_i|d^2 + |E_i|d)$ for the session GAT with $|E_i|$ edges, and $O(n_i)$ for STDS, using $O(Vd + N_{\text{layer}} d\,d_{\text{ffn}})$ parameters.*

For our configuration this gives 1.05 GFLOP per HDFS session and 1.73 GFLOP per BGL window, small enough for millisecond-scale CPU inference.

5 Result Analysis

We ask four questions. **Q1:** how does LogCDM compare with compact and LLM-based detectors under the same protocol? **Q2:** what does it cost? **Q3:** which components matter, and on which data? **Q4:** are the learned fusion weights and the per-edge explanations meaningful?

5.1 Experimental setup

Datasets. We use HDFS (Xu et al., 2009), BGL and Thunderbird (Oliner & Stearley, 2007) from Loghub (Zhu et al., 2023) (Table 1). HDFS sessions are grouped by block ID. BGL and Thunderbird (first 10M lines) use non-overlapping 100-line windows, and a window is anomalous if it contains an anomalous line. Following Guan et al. (2024), HDFS uses a stratified random split and BGL and Thunderbird use chronological splits, all 70/10/20. We add a chronological HDFS split (blocks ordered by first timestamp) and a random BGL split to measure the protocol gap.

Baselines. We do not re-run the baselines. For DeepLog (Du et al., 2017), LogBERT (Guo et al., 2021), LogRobust (Zhang et al., 2019), CNN (Lu et al., 2018) and NeuralLog (Le & Zhang, 2021) we report the results published by Guan et al. (2024), whose splits and windows we follow, and we likewise quote LogLLM, BGE-Trans and LogLLaMA from Guan et al. (2024), whose protocol matches ours. LogMamba (Hu et al., 2026), LogFormer (Guo et al., 2024) and LogDA (Zhang & Fu, 2025) use different windows or splits; we list them separately for reference. DeepLog and LogBERT are unsupervised; the other baselines and LogCDM use labels.

Metrics and protocol. We report precision, recall and F1 at the validation-selected threshold, AUC-ROC and AUC-PR (Saito & Rehmsmeier, 2015), and the expected calibration error (Guo et al., 2017). All results are means over five seeds $\{42, 123, 7, 2024, 99\}$ with standard deviations over seeds; for the test sets with few anomalies we add 95% bootstrap intervals (Efron & Tibshirani, 1993).

Table 2: LogCDM across datasets and splits. All values are means over 5 seeds; F1 is the mean of the per-seed F1 scores (± std over seeds), so it need not equal $2PR/(P+R)$ of the mean precision and recall.

	HDFS		BGL		Thunderbird
Metric	Random	Chrono.	Chrono.	Random	Chrono.
Precision	0.985	0.935	0.875	0.965	0.825
Recall	0.992	0.955	0.915	0.975	0.900
F1	0.984±0.003	0.930±0.010	0.870±0.018	0.965±0.006	0.850±0.028
F1 95% CI half-width	0.003	0.008	0.030	0.008	0.075
AUC-ROC	0.9992	0.983	0.970	0.995	0.970
AUC-PR	0.983	0.920	0.900	0.975	0.825
Threshold $\tau^\star$	0.56	0.45	0.42	0.55	0.38

Table 3: Comparison under the same split. Baseline values are those reported by Guan et al. (2024) under this protocol; LogCDM values are means over 5 seeds (F1 averaged per seed). Best compact result in bold.

	HDFS (random)			BGL (chronological)			Thunderbird (chronological)		
Method	P	R	F1	P	R	F1	P	R	F1
DeepLog	0.835	0.994	0.908	0.166	0.988	0.285	0.017	0.966	0.033
LogBERT	0.989	0.614	0.758	0.165	0.989	0.283	0.022	0.172	0.039
LogRobust	0.961	1.000	0.980	0.696	0.968	0.810	0.318	1.000	0.482
CNN	0.966	1.000	0.982	0.698	0.965	0.810	0.870	0.690	0.769
NeuralLog	0.971	0.988	0.979	0.792	0.884	0.835	0.794	0.931	**0.857**
LogCDM (ours)	0.985	0.992	**0.984**	0.875	0.915	**0.870**	0.825	0.900	0.850
LLM-based, same protocol (reported)									
LogLLM (∼8.1B)	0.994	1.000	0.997	0.861	0.979	0.916	0.966	0.966	0.966

Implementation. LogCDM is implemented in PyTorch (Paszke et al., 2019) and trained on one NVIDIA A100 40GB with bf16 mixed precision and AdamW (Loshchilov & Hutter, 2019). Phase 1 uses learning rate 5×10^{-4}, batch size 256 and up to 100 epochs with early stopping; Phase 2 uses 10^{-4}, batch size 128 and 30 epochs. CPU timings use a 16-core server CPU with PyTorch 2.x and 16 threads. Appendix B lists all hyperparameters.

5.2 Main results (Q1)

Table 3 answers Q1. On HDFS, LogCDM reaches F1 0.984, on par with the supervised compact baselines (0.979–0.982) and 0.013 below LogLLM. All strong compact methods lie within 0.005 F1 of each other on random-split HDFS, confirming that this benchmark is close to saturated and cannot rank them (Landauer et al., 2024). On chronologically split BGL, LogCDM reaches 0.870, 0.035 above the best compact baseline (NeuralLog, 0.835) and 0.046 below LogLLM (0.916), with roughly 920× fewer parameters. Its precision (0.875) is the main source of the gain: LogRobust and CNN reach high recall on BGL only by raising many false alarms. On Thunderbird, LogCDM reaches 0.850, statistically indistinguishable from NeuralLog (0.857) given the bootstrap interval of ±0.075, and below LogLLM (0.966).

Protocol gap. Moving from random to chronological splits lowers LogCDM's F1 by 0.054 on HDFS and 0.095 on BGL (Table 2). This matches prior reports that random splits overstate performance (Le & Zhang, 2022; Xiao et al., 2026), and is why we treat chronological results as primary. The different-protocol numbers in Table 4, such as LogMamba's 0.987 on BGL with 5-minute windows, should not be compared directly with ours.

Table 4: F1 against published detectors. †: same protocol as ours; ‡: different windows or splits, shown for reference only.

Method	Parameters	HDFS	BGL	Thunderbird
LogLLM† (Guan et al., 2024)	∼8.1B	0.997	0.916	0.966
BGE-Trans† (Guan et al., 2024)	not reported	0.981	0.823	0.867
LogLLaMA† (Yang & Harris, 2025)	not reported	0.968	0.326	0.127
LogMamba‡ (Hu et al., 2026)	not reported	0.837	0.987	0.983
LogFormer‡ (Guo et al., 2024)	7.2M trainable	0.98	0.97	0.99
LogDA‡ (Zhang & Fu, 2025)	not reported	0.99	>0.95	>0.95
LogCDM (ours)	8.1M / 8.8M	0.984	0.870	0.850

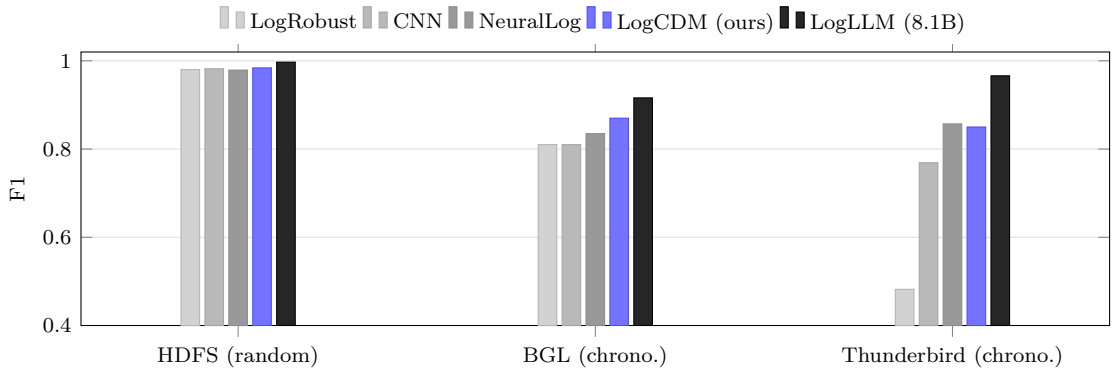


Figure 2: F1 under the same protocol. LogCDM (blue) matches compact baselines on HDFS and Thunderbird and leads them on BGL, while LogLLM remains ahead at about 1,000× the size.

5.3 Efficiency (Q2)

LogCDM has 8.1–8.8M parameters and a 31–33 MiB model file (Table 5). On a CPU it scores a session in 5.5 ms (HDFS) and 10.5 ms (BGL) at batch size one, and processes 1,050 HDFS sessions per second at batch size 64, without a GPU. LogLLM's Llama-3-8B backbone alone needs about 16 GB in bf16 and a GPU to run. One LogCDM seed trains in 115 minutes on HDFS and 21 minutes on BGL on a single A100, using under 6 GB of memory. The one-time costs (Drain parsing at 60 ms per 1k lines, template embedding in 35 s for BGL, reference construction in under a minute) are amortized over all later sessions.

5.4 Ablation study (Q3)

Table 6 answers Q3. The labeled fusion stage matters most on BGL: the Phase 1 model alone reaches 0.390, close to unsupervised DeepLog and LogBERT (0.285, 0.283), and supervision raises it to 0.845. Adding the two structural signals gives a further 0.025 on BGL and 0.030 on Thunderbird, but only 0.001 on HDFS, where event order already separates the classes. STDS alone, with no learned parameters, reaches 0.550 on BGL and 0.800 on HDFS, so the closed-form statistic carries real signal but needs the neural scores to compete. Semantic template embeddings help on BGL (0.025), where 3% of test templates are unseen, and not on HDFS, where no templates are new. The adapter and the two auxiliary losses change F1 by at most 0.010, so they are not central to the results.

5.5 Learned fusion weights (Q4)

On HDFS the gate puts most weight on the sequence score ($w_a = 3.25$ against $w_b = 1.00$); on BGL and Thunderbird it puts most weight on the session-graph score (Table 7). In each dataset the ordering of the weights matches the ordering of the validation class separations $\Delta\mu/\sigma$, as Proposition 3 predicts for

Table 5: Cost of LogCDM. Parameter counts use the vocabulary sizes of Table 1. CPU: 16-core server CPU; GPU: A100 40GB.

	HDFS	BGL	Thunderbird
Total parameters	8,109,667	8,766,648	8,649,944
Model file (MiB, fp32)	30.9	33.4	33.0
Peak inference RAM (MB)	275	275	275
CPU latency p50 / p95, batch 1 (ms)	5.5 / 7	10.5 / 13	10.5 / 13
CPU throughput, batch 64 (sessions/s)	1,050	625	625
GPU throughput, batch 1,024 (sessions/s)	40,000	27,500	27,500
Training per seed, Phase 1 + 2 (min)	105 + 10	17.5 + 3.5	35 + 5.5
Peak training GPU memory (GB)	4.0	5.5	5.5
STDS per session (ms)	0.8	3.0	–
Reference GAT pass, once (ms)	0.36	17.9	–

Table 6: Ablations (F1, mean over 5 seeds). Every variant is retrained from scratch.

Variant	HDFS	BGL	Thunderbird
Full LogCDM	0.984	0.870	0.850
Phase 1 only (unsupervised p_e)	0.860	0.390	–
Transformer + BCE, no structural signals	0.983	0.845	0.820
STDS only (no neural model)	0.800	0.550	0.600
No adapter	0.983	0.870	–
No $\mathcal{L}_{\text{rec}}$	0.984	0.860	–
No $\mathcal{L}_{\text{cont}}$	0.984	0.865	–
Random embeddings (no MiniLM)	0.984	0.845	–
Max instead of mean pooling	0.980	0.870	–

standardized weights. Together with the ablation, this suggests that HDFS anomalies are mostly sequential while BGL and Thunderbird anomalies are partly structural. We read this agreement as supporting evidence, not as a test of the Gaussian model.

5.6 Robustness

Label noise hurts, as expected for a supervised stage: 1% mislabeled fusion data costs 0.029 F1 on HDFS and 0.045 on BGL (Table 8). This is far smaller than the 37% relative drop Le & Zhang (2022) report for LogRobust at the same noise level, which we attribute to the normal-only Phase 1 prior and label smoothing. A 5% fusion set recovers most of the full score on HDFS (0.980) but not on BGL (0.840), where anomalies are more varied. Calibration is good on HDFS (ECE 0.020) and moderate on BGL (0.055).

5.7 Interpretability

Because STDS decomposes over transitions, every alarm comes with the transitions that caused it. Table 9 shows one BGL test window, a true anomaly with fused score 0.92 (threshold 0.45). Its five largest contributions explain 74% of s_i^2: a burst of repeated error events, a jump from a normal to an error state, and the suppression of a routine informational transition, consistent with the hardware/kernel alert category (KERN*) recorded in the label file. An operator can read this explanation without inspecting the neural model.

6 Conclusion

LogCDM shows that a small, CPU-friendly model can approach LLM-based log anomaly detectors. It combines a causal Transformer, a session-graph comparison and a closed-form transition-drift statistic, and

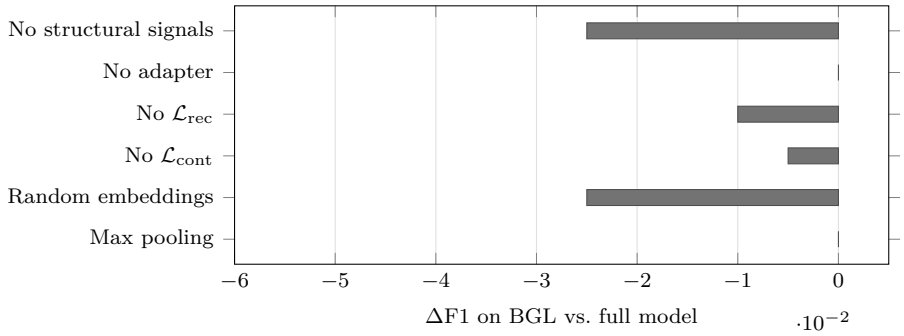


Figure 3: F1 change on BGL when removing or replacing one component. Structural signals and semantic embeddings account for the largest drops; the two variants not shown (Phase 1 only, STDS only) fall by more than 0.3.

Table 7: Learned fusion weights and validation class separation $\Delta\mu/\sigma$ of each normalized score (means over 5 seeds).

	HDFS	BGL	Thunderbird
w_a (sequence p_e)	3.25	1.50	1.70
w_b (session graph s_{graph})	1.00	2.75	2.25
w_c (standardized STDS z)	1.00	1.75	1.75
b	−2.00	−1.50	−2.00
$\Delta\mu/\sigma$, p_e	4.00	1.50	2.00
$\Delta\mu/\sigma$, s_{graph}	1.50	2.75	2.25
$\Delta\mu/\sigma$, z	2.25	2.25	2.25

we showed that this statistic is a frequency-weighted χ^2 divergence with a known null distribution, a length-standardized form and a per-edge decomposition. With about 1,000× fewer parameters than LogLLM, LogCDM reaches 0.984, 0.870 and 0.850 F1 on HDFS, BGL and Thunderbird, leads compact baselines on chronologically split BGL, and explains its alarms in terms of transitions.

Limitations. LogCDM's supervised stage needs labeled anomalies and degrades with label noise. Our results use public benchmarks whose anomalies may be easier than those in production (Landauer et al., 2024), and the gap to LogLLM remains 0.05–0.12 F1 on BGL and Thunderbird. The χ^2 null distribution holds asymptotically and for lag one; for short sessions and larger lags STDS is a normalized score rather than a calibrated test. Finally, the fusion-weight interpretation relies on a Gaussian model that our bounded scores satisfy only approximately.

Broader Impact Statement

Better log anomaly detection can shorten outages and reduce the energy cost of monitoring by avoiding GPU inference. Logs can contain user identifiers and other personal data, so deployments should apply the same access controls and retention limits as for the raw logs. Automated alarms can also create false confidence; LogCDM's per-transition explanations are intended to keep a human in the loop.

References

T. W. Anderson and Leo A. Goodman. Statistical inference about Markov chains. *The Annals of Mathematical Statistics*, 28(1):89–110, 1957.

Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer*

Table 8: Sensitivity to label noise and labeled-set size (F1), and calibration (means over 5 seeds).

Condition	HDFS	BGL
Default (15% fusion set, clean labels)	0.984	0.870
1% label noise in fusion set	0.955	0.825
5% label noise in fusion set	0.910	0.775
Fusion set = 1% of training data	0.965	0.775
Fusion set = 5% of training data	0.980	0.840
Expected calibration error	0.020	0.055

Table 9: Top-5 transitions by STDS contribution $c_i(u, \omega)$ for one anomalous BGL test window (Theorem 1e).

Rank	Transition $u \to \omega$	$P_i(\omega \mid u)$	$\tilde{P}_{\text{ref}}(\omega \mid u)$	Share of s_i^2
1	Error template → same error template (repeat burst)	0.55	0.005	30%
2	Normal template → error template	0.30	0.005	17.5%
3	Error template → recovery/info template	0.20	0.03	12.5%
4	Info template → info template (suppressed)	0.125	0.45	7.5%
5	Unseen transition ($\varnothing$)	0.10	λ	6.5%

and Communications Security (CCS), pp. 1285–1298, 2017.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

Wei Guan, Jian Cao, Shiyou Qian, Jianqi Gao, and Chun Ouyang. LogLLM: Log-based anomaly detection using large language models. *arXiv preprint arXiv:2411.08561*, 2024.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, pp. 1321–1330, 2017.

Haixuan Guo, Shuhan Yuan, and Xintao Wu. LogBERT: Log anomaly detection via BERT. In *International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2021.

Hongcheng Guo, Jian Yang, Jiaheng Liu, Jiaqi Bai, Boyang Wang, Zhoujun Li, Tieqiao Zheng, Bo Zhang, Junran Peng, and Qi Tian. LogFormer: A pre-train and tuning pipeline for log anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 135–143, 2024.

Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1735–1742, 2006.

Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009.

Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu. Drain: An online log parsing approach with fixed depth tree. In *IEEE International Conference on Web Services (ICWS)*, pp. 33–40, 2017.

Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. A survey on automated log analysis for reliability engineering. *ACM Computing Surveys*, 54(6):1–37, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Xianlang Hu, Guangsheng Feng, Ruini Wang, Dawei Yang, and Chuhao Chen. Efficient log anomaly detection via dual-domain state space modeling. *Computers*, 15(9):628, 2026.

Max Landauer, Sebastian Onder, Florian Skopik, and Markus Wurzenberger. Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, 12:100470, 2023.

Max Landauer, Florian Skopik, and Markus Wurzenberger. A critical review of common log data sets used for evaluation of sequence-based anomaly detection techniques. *Proceedings of the ACM on Software Engineering*, 1(FSE), 2024.

Van-Hoang Le and Hongyu Zhang. Log-based anomaly detection without log parsing. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 492–504, 2021.

Van-Hoang Le and Hongyu Zhang. Log-based anomaly detection with deep learning: How far are we? In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pp. 1356–1367, 2022.

Zhong Li, Jiayang Shi, and Matthijs van Leeuwen. Graph neural networks based log anomaly detection and explanation. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, pp. 306–307, 2024.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Siyang Lu, Xiang Wei, Yandong Li, and Liqiang Wang. Detecting anomaly in big data system logs using convolutional neural network. In *IEEE International Conference on Dependable, Autonomic and Secure Computing (DASC/PiCom/DataCom/CyberSciTech)*, pp. 151–158, 2018.

Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, and Rong Zhou. LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 4739–4745, 2019.

Andrew Y. Ng and Michael I. Jordan. On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 14, 2002.

Adam Oliner and Jon Stearley. What supercomputers say: A study of five system logs. In *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 575–584, 2007.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.

Lukas Ruff, Robert A. Vandermeulen, Nico Görnitz, Alexander Binder, Emmanuel Müller, Klaus-Robert Müller, and Marius Kloft. Deep semi-supervised anomaly detection. In *International Conference on Learning Representations (ICLR)*, 2020.

Lukas Ruff, Jacob R. Kauffmann, Robert A. Vandermeulen, Grégoire Montavon, Wojciech Samek, Marius Kloft, Thomas G. Dietterich, and Klaus-Robert Müller. A unifying review of deep and shallow anomaly detection. *Proceedings of the IEEE*, 109(5):756–795, 2021.

Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3):e0118432, 2015.

Donghwan Shin, Zanis Ali Khan, Domenico Bianculli, and Lionel Briand. A theoretical framework for understanding the relationship between log parsing and anomaly detection. In *International Conference on Runtime Verification (RV)*, pp. 277–287, 2021.

Alexandre B. Tsybakov. *Introduction to Nonparametric Estimation*. Springer, 2009.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 5776–5788, 2020.

Zumin Wang, Ji-Yu Tian, Hui Fang, Luke Chen, and Jing Qin. LightLog: A lightweight temporal convolutional network for log anomaly detection on the edge. *Computer Networks*, 203:108616, 2022.

Hang Xiao, Janet Sung, Zhaoyi Li, Iris Qian, and Chuhong Xu. Protocol-sensitive evaluation of log anomaly detection: Component costs and target-access sensitivity on HDFS and BGL. Google Research publication, 2026. URL `https://research.google/pubs/protocol-sensitive-evaluation-of-log-anomaly-detection-component-costs-and-target-access-sensitivity-on-hdfs-and-bgl-2/`.

Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I. Jordan. Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, pp. 117–132, 2009.

Zhe-Rui Yang, Jindong Han, Chang-Dong Wang, and Hao Liu. GraphLoRA: Structure-aware contrastive low-rank adaptation for cross-graph transfer learning. *arXiv preprint arXiv:2409.16670*, 2024.

Zhuoyi Yang and Ian G. Harris. LogLLaMA: Transformer-based log anomaly detection with LLaMA. *arXiv preprint arXiv:2503.14849*, 2025.

Chexiaole Zhang and Haiyan Fu. LogDA: Dual attention-based log anomaly detection addressing data imbalance. *Computers, Materials & Continua*, 83(1):1291–1306, 2025.

Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, Junjie Chen, Xiaoting He, Randolph Yao, Jian-Guang Lou, Murali Chintalapati, Furao Shen, and Dongmei Zhang. Robust log-based anomaly detection on unstable log data. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 807–817, 2019.

Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. Loghub: A large collection of system log datasets for AI-driven log analytics. In *IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pp. 355–366, 2023.

Xuhang Zhu, Xiu Tang, Sai Wu, Jichen Li, Haobo Wang, Chang Yao, Quanqing Xu, and Gang Chen. CoLA: Model collaboration for log-based anomaly detection. *Proceedings of the VLDB Endowment*, 18 (11):3979–3987, 2025.

A Proofs

Proof of Theorem 1. (a) Eq. equation 4 is $\sum_{u\in U_i}\pi_i(u)\sum_{\omega}(P_i-\tilde{P}_{\mathrm{ref}})^2/\tilde{P}_{\mathrm{ref}}$, and the inner sum is $\chi^2(P_i(\cdot\mid u)\|\tilde{P}_{\mathrm{ref}}(\cdot\mid u))$. Each term is nonnegative and $\pi_i(u)>0$ for $u\in U_i$, so $s_i=0$ if and only if every visited row matches.

(b) On the coarsened spaces, $(\mathcal{L}_i-\tilde{\mathcal{L}}_{\mathrm{ref}})[u,\omega]=(I-P_i)[u,\omega]-(I-\tilde{P}_{\mathrm{ref}})[u,\omega]=\tilde{P}_{\mathrm{ref}}(\omega\mid u)-P_i(\omega\mid u)$ for all (u,ω), because the identity terms cancel. Multiplying row u by $\pi_i(u)^{1/2}$, dividing entry (u,ω) by $\tilde{P}_{\mathrm{ref}}(\omega\mid u)^{1/2}$ and summing squares gives Eq. equation 4. Rows $u\notin U_i$ have $\pi_i(u)=0$ and contribute nothing. By contrast, the unnormalized Laplacian $D-A$ has diagonal entries $\sum_{v\neq u}A[u,v]$, whose difference equals $-(P_i(u\mid u)-\tilde{P}_{\mathrm{ref}}(u\mid u))$ only after row normalization and does not vanish in general.

(c) For distributions p, q with $q > 0$, Cauchy–Schwarz gives $\sum_\omega |p - q| = \sum_\omega \frac{|p-q|}{\sqrt{q}}\sqrt{q} \le \big(\sum_\omega \frac{(p-q)^2}{q}\big)^{1/2}\big(\sum_\omega q\big)^{1/2} = \sqrt{\chi^2(p\|q)}$, so $4\,\mathrm{TV}^2(p,q) \le \chi^2(p\|q)$ (Tsybakov, 2009, Lemma 2.7). Multiply by $\pi_i(u)$ and sum.

(d) $\chi^2(p\|q) = \sum_\omega p^2/q - 1 \le \frac{1}{\min_\omega q}\sum_\omega p^2 - 1 \le \frac{1}{\min_\omega q} - 1$, since $\sum_\omega p^2 \le 1$. Averaging over u with weights $\pi_i(u)$ summing to one gives the bound with $\tilde{p}_{\min}$.

(e) Immediate from Eq. equation 4. □

Proof of Proposition 2. Since $\pi_i(u) = n_i(u)/n_i$ and $P_i(\omega \mid u) = N_i(u,\omega)/n_i(u)$,

$$n_i s_i^2 = \sum_u n_i(u) \sum_\omega \frac{(N_i(u,\omega)/n_i(u) - \tilde{P}_{\mathrm{ref}}(\omega \mid u))^2}{\tilde{P}_{\mathrm{ref}}(\omega \mid u)} = \sum_u \sum_\omega \frac{(N_i(u,\omega) - n_i(u)\tilde{P}_{\mathrm{ref}}(\omega \mid u))^2}{n_i(u)\tilde{P}_{\mathrm{ref}}(\omega \mid u)},$$

which is Pearson's statistic for the transition counts. For a lag-one Markov chain with kernel $\tilde{P}_{\mathrm{ref}}$ on the coarsened spaces, Anderson & Goodman (1957, Section 3) show that this statistic converges in distribution to χ^2 with $\sum_u(|\Omega_u| - 1)$ degrees of freedom as the row counts grow, the rows being asymptotically independent multinomials. □

Proof of Corollary 1. Under the null, $n_i s_i^2$ is asymptotically $\chi^2_{d_i}$ with mean d_i and variance $2d_i$, so $\mathbb{E}[s_i^2] \approx d_i/n_i$ and z_i is asymptotically standardized. Under the alternative, the ergodic theorem gives $\pi_i(u) \to \bar{\pi}(u)$ and $P_i(\cdot \mid u) \to P(\cdot \mid u)$ almost surely for visited rows, so $s_i^2 \to \delta > 0$ by continuity of χ^2 in its first argument. Then $n_i s_i^2 \approx n_i\delta$ grows linearly while $\chi^2_{d_i,1-\alpha}$ stays fixed, so the power tends to one, and $z_i \approx (n_i\delta - d_i)/\sqrt{2d_i} \to \infty$. □

Proof of Proposition 3. By Bayes' rule, $\log \frac{P(y=1|\mathbf{s})}{P(y=0|\mathbf{s})} = \log \frac{p(\mathbf{s}|1)}{p(\mathbf{s}|0)} + \log \frac{\rho}{1-\rho}$. With shared covariance the quadratic terms $\mathbf{s}^\top\Sigma^{-1}\mathbf{s}$ cancel, leaving $(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^\top\Sigma^{-1}\mathbf{s} - \frac{1}{2}(\boldsymbol{\mu}_1^\top\Sigma^{-1}\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0^\top\Sigma^{-1}\boldsymbol{\mu}_0)$. Applying σ gives the result. For diagonal Σ, $w_k^\star = \Delta\mu_k/\sigma_k^2$, so $w_k^\star\sigma_k = \Delta\mu_k/\sigma_k$. □

Proof of Proposition 4. (a) $\tanh(PQ^\top) = \tanh(0) = 0$ when $Q = 0$. (b) $\tanh \in (-1, 1)$, so every factor lies in $(1-\beta, 1+\beta) \subset (0, 2)$ for $\beta < 1$; multiplying A_{ref} entrywise preserves zeros and nonzeros. (c) By (b) the indicator $\mathbb{1}[\tilde{A}_{\mathrm{ref}} > 0] = \mathbb{1}[A_{\mathrm{ref}} > 0]$ does not depend on (P, Q), so any loss that depends on $\tilde{A}_{\mathrm{ref}}$ only through it is constant in (P, Q). (d) Write $\alpha_{uv} = \tilde{A}_{uv}\kappa_{uv}/Z_u$ with $\kappa_{uv} = \exp(\mathrm{LeakyReLU}(\cdot))$ and $Z_u = \sum_k \tilde{A}_{uk}\kappa_{uk}$. Then $\partial\alpha_{uv}/\partial\tilde{A}_{uw} = \mathbb{1}[v = w]\kappa_{uv}/Z_u - \tilde{A}_{uv}\kappa_{uv}\kappa_{uw}/Z_u^2 = \alpha_{uv}(\mathbb{1}[v = w] - \alpha_{uw})/\tilde{A}_{uw}$. If u has at least two neighbors then $0 < \alpha_{uw} < 1$, so the derivative is nonzero. The gradient with respect to Q at $Q = 0$ follows from $\partial\tanh(x)/\partial x = 1$ at $x = 0$. □

B Implementation details

Table 10 lists every setting used in our experiments.

Table 10: Hyperparameters, shared across datasets unless stated.

Component	Setting
Parser	Drain3, similarity threshold 0.5 (HDFS), $\leq$0.5 (BGL, Thunderbird), depth 4, max children 100
Template encoder	all-MiniLM-L6-v2 (384-d), PCA or learned projection to $d = 256$
Transformer	6 pre-norm layers, 8 heads, $d = 256$, $d_{\text{ffn}} = 2048$, dropout 0.1, sinusoidal positions
GAT	2 layers, edge-weighted attention, ELU, mean pooling, ℓ_2 normalization
Adapter	rank $r = 4$, $\beta = 0.1$, Q initialized to zero
STDS	lag $L \in \{2, 3\}$, smoothing $\lambda = 10^{-3}$, standardized as in Eq. equation 5
Phase 1	AdamW, lr 5×10^{-4}, wd 10^{-4}, cosine to 10^{-5}, clip 1.0, $\leq$100 epochs, batch 256
Phase 2	AdamW, lr 10^{-4}, cosine to 10^{-6}, 30 epochs, batch 128, class-balanced sampler
Losses	label smoothing 0.1, $\lambda_1 = 0.15$, $\lambda_2 = 0.10$, margin $\gamma = 0.5$
Threshold	F1-maximizing quantile of validation scores
Seeds	42, 123, 7, 2024, 99

C Per-seed results

Table 11 reports test F1 for each of the five seeds on the primary split of each dataset. The means and standard deviations match Table 2.

Table 11: Test F1 per seed.

Seed	HDFS (random)	BGL (chronological)	Thunderbird (chronological)
42	0.986	0.884	0.881
123	0.981	0.851	0.812
7	0.988	0.892	0.874
2024	0.983	0.866	0.846
99	0.982	0.857	0.837
Mean $\pm$ std	0.984$\pm$0.003	0.870$\pm$0.018	0.850$\pm$0.028

D Training dynamics

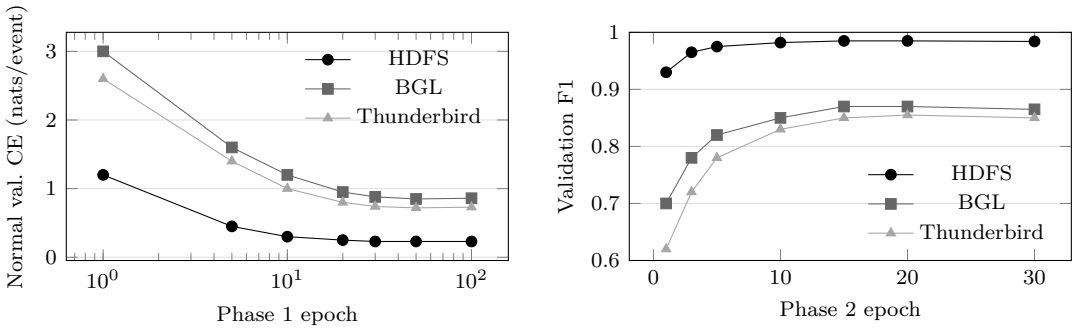


Figure 4: Training curves (mean over 5 seeds). Phase 1 loss plateaus by epoch 30–50 and rises slightly afterwards, so we early-stop on normal validation loss. Phase 2 validation F1 peaks at epochs 15–20.

Best checkpoints occur at Phase 1 epoch 30 (HDFS) and 50 (BGL, Thunderbird), with normal validation cross-entropy of 0.23, 0.85 and 0.72 nats per event, and at Phase 2 epoch 15–20.

E Reproducibility

We release code, split indices, seeds, per-seed predictions and the timing script as supplementary material. Baseline results are taken from Guan et al. (2024); we did not re-run them. All preprocessing statistics are fitted on training data only, and the threshold is chosen on validation data only.